Code Explanation: comparative_text_classification.py [newhorizon...epoint.com]

Aim

This program compares three text classification algorithms:

1. **Naive Bayes**
2. **Support Vector Machine (SVM)**
3. **Centroid-Based Classifier**

using the same dataset and evaluates them using:

- Accuracy
- Precision
- Recall
- F1-Score

[newhorizon...epoint.com]

---

1. Import Required Libraries

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics.pairwise import cosine_similarity
 import numpy as np
```

Explanation

TF-IDF Vectorizer

```
TfidfVectorizer
```

Converts text documents into numeric vectors.

MultinomialNB

```
MultinomialNB
```

Implements the Naive Bayes classifier.

SVC

```
SVC
```

Implements Support Vector Machine.

---

train_test_split

```
train_test_split
```

Splits data into training and testing sets.

Metrics

```
accuracy_score()
precision_score()
recall_score()
f1_score()
```

Used to evaluate classifier performance.

Cosine Similarity

```
cosine_similarity()
```

Measures similarity between two vectors.

NumPy

```
import numpy as np
```

Used for numerical operations.

[newhorizon...epoint.com]

---

2. Create Dataset

```
documents = [
    "machine learning and artificial intelligence",
    "deep learning neural networks",
```

```
    "natural language processing",
    "data science and machine learning",
    "artificial intelligence systems",
    "neural network applications",
    "football world cup tournament",
    "cricket match highlights",
     "soccer championship players",
     "basketball league tournament",
     "tennis grand slam match",
     "football coaching techniques"
]
```

Explanation

This is the collection of text documents.

Technology Documents:

```
machine learning
deep learning
AI
neural networks
```

Sports Documents:

```
football
cricket
soccer
basketball
tennis
```

[newhorizon...epoint.com]

---

3. Create Labels

```
labels = [
    "Technology",
    "Technology",
    "Technology",
    "Technology",
    "Technology",
    "Technology",
    "Sports",
```

```
    "Sports",
    "Sports",
    "Sports",
    "Sports",
    "Sports"
]
```

Explanation

Each document has a class label.

Example:

```
machine learning → Technology

football world cup → Sports
```

These labels are used for supervised learning.

[newhorizon...epoint.com]

---

4. TF-IDF Feature Extraction

```
vectorizer = TfidfVectorizer()
```

Creates a TF-IDF object.

```
X = vectorizer.fit_transform(documents)
```

What happens?

The program:

1. Splits documents into words.
2. Computes TF-IDF scores.
3. Converts text into numerical vectors.

Example:

Word	TF-IDF
machine	0.65
learning	0.71
football	0.78

Machine learning algorithms cannot understand text directly.

TF-IDF converts the text into numbers.

[newhorizon...epoint.com]

---

5. Split Data

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    labels,
    test_size=0.30,
    random_state=42,
    stratify=labels
)
```

Explanation

The dataset is divided into:

```
70% Training Data
30% Testing Data
```

---

random_state=42

Ensures the same split every time.

---

stratify=labels

Maintains equal proportions of:

```
Technology
Sports
```

in both train and test sets.

[newhorizon...epoint.com]

---

6. Naive Bayes Classifier

```
nb_model = MultinomialNB()
```

Creates a Naive Bayes model.

```
nb_model.fit(X_train, y_train)
```

Training

The model learns patterns such as:

```
learning → Technology

football → Sports
```

```
nb_predictions = nb_model.predict(X_test)
```

Predicts categories for unseen documents.

Example:

```
football coaching techniques
```

Output:

```
Sports
```

[newhorizon...epoint.com]

---

7. Support Vector Machine (SVM)

```
svm_model = SVC(kernel='linear')
```

Creates an SVM classifier.

Linear Kernel

```
kernel='linear'
```

Used because text classification data is usually high-dimensional.

```
svm_model.fit(X_train, y_train)
```

Trains the model.

```
svm_predictions = svm_model.predict(X_test)
```

Predicts classes for test documents.

[newhorizon...epoint.com]

---

8. Centroid Classifier

Create Categories

```
categories = list(set(y_train))
```

Output:

```
['Technology', 'Sports']
```

[newhorizon...epoint.com]

Create Empty Dictionary

```
centroids = {}
```

Used to store centroid vectors.

Calculate Category Centroids

```
for category in categories:
```

Loops through:

```
Technology
Sports
```

Find Matching Documents

```
indices = [
    i for i in range(len(y_train))
    if y_train[i] == category
]
```

Example:

```
Technology → [0,1,2,3]
```

Calculate Mean Vector

```
centroid = np.asarray(
    X_train[indices].mean(axis=0)
)
```

Computes average TF-IDF vector.

This becomes the centroid.

Example:

```
Technology Centroid
```

```
Average(
Machine Learning,
Deep Learning,
AI,
NLP
)
```

[newhorizon...epoint.com]

Store Centroid

```
centroids[category] = centroid
```

Example:

```
{
  'Technology': vector1,
  'Sports': vector2
}
```

[newhorizon...epoint.com]

---

9. Predict Using Centroids

```
centroid_predictions = []
```

Stores predictions.

```
for doc in X_test:
```

Processes each test document.

```
best_category = None
best_similarity = -1
```

Initialize variables.

Compare with Each Centroid

```
similarity = cosine_similarity(
```

```
    doc,
    centroids[category]
)[0][0]
```

Formula:

Where:

- A = Test document
- B = Category centroid

Example

```
Technology similarity = 0.82

Sports similarity = 0.12
```

Prediction:

```
Technology
```

```
if similarity > best_similarity:
```

Keeps the highest similarity.

```
centroid_predictions.append(best_category)
```

Stores final prediction.

[newhorizon...epoint.com]

---

10. Evaluation Function

```
def evaluate(name, actual, predicted):
```

A reusable function used to evaluate all three models.

[newhorizon...epoint.com]

Accuracy

```
accuracy_score(actual, predicted)
```

Formula:

Example:

```
4 Correct
4 Total

Accuracy = 1.0
```

Precision

```
precision_score()
```

Formula:

Measures prediction correctness.

Recall

```
recall_score()
```

Formula:

Measures how many actual positives were identified.

F1 Score

```
f1_score()
```

Formula:

Balances precision and recall.

[newhorizon...epoint.com]

---

11. Display Results

```
print("\nActual Test Labels:")
print(y_test)
```

Displays actual classes.

```
evaluate(
    "Naive Bayes",
    y_test,
    nb_predictions
)
```

Evaluates Naive Bayes.

```
evaluate(
    "Support Vector Machine",
    y_test,
    svm_predictions
)
```

Evaluates SVM.

```
evaluate(
    "Centroid Classifier",
    y_test,
    centroid_predictions
)
```

Evaluates Centroid Classifier.

[newhorizon...epoint.com]

Flow of the Program

```
Documents
      ↓
TF-IDF Vectorization
      ↓
Train-Test Split
      ↓
  ┌──────────────┐
  │ Naive Bayes  │
  └──────────────┘
       ↓
   ┌──────────────┐
   │     SVM      │
   └──────────────┘
       ↓
   ┌──────────────┐
   │  Centroid    │
   └──────────────┘
       ↓
```

```
Predictions
      ↓
Accuracy
Precision
Recall
F1 Score
```

Expected Result

The program successfully compares **Naive Bayes**, **SVM**, and **Centroid-Based Classification** on the same dataset and evaluates their performance using **Accuracy, Precision, Recall, and F1-Score**. comparative_text_classification.py [newhorizon...epoint.com]